

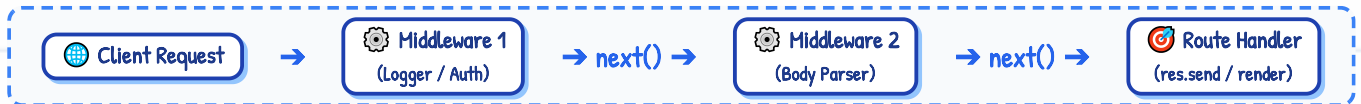


07. Express Middleware Fundamentals



What is Middleware?

Middleware functions execute in the middle of the **Request-Response cycle**. They have access to **req**, **res**, and the **next** function in the application's middleware stack.



Middleware Capabilities

- Execute any code (e.g. logging metrics, authentication tokens).
- Modify request and response objects (e.g. attaching `req.user = decodedToken`).
- End the request-response cycle directly (e.g. sending 401 Unauthorized).
- Pass control to the next middleware in the stack via **next()**.



Custom Logger Middleware Example

```
index.js - Custom Logger Middleware

app.use((req, res, next) => {
  req.responseTime = new Date(Date.now()).toString();
  console.log(`[${req.method}] ${req.path} at ${req.responseTime}`);
  next(); // 🚨 Essential: Without next(), request will hang indefinitely!
});
```



NEVER FORGET `next()`: If a middleware neither sends a response nor calls `next()`, the client request remains hanging!



Error-Handling Middleware & Custom Errors



Error-Handling Middleware: 4 Arguments Signature

Error-handling middleware functions are defined with exactly 4 parameters: **(err, req, res, next)**. Express uses this 4-argument signature to identify error handlers.

next(): Proceeds to the next regular middleware or route handler in line.

next(err): Bypasses all remaining route handlers and jumps directly to Error Middlewares!



Custom Error Class (ExpressError)

Best practice for maintaining custom HTTP status codes and structured messages:

```
ExpressError.js - Custom Error Class

class ExpressError extends Error {
  constructor(status, message) {
    super();
    this.status = status;
    this.message = message;
  }
}
module.exports = ExpressError;
```



Global Error Handler & 404 Route

```
index.js - Error Handling Setup

// 1. 404 Catch-All Route
app.all("*", (req, res, next) => {
  next(new ExpressError(404, "Page Not Found!"));
});

// 2. Global Error Handling Middleware
app.use((err, req, res, next) => {
  let { status = 500, message = "Something Went Wrong!" } = err;
  res.status(status).render("error.ejs", { message });
});
```